

Rapport de Travaux Pratiques

Configuration d'un Reverse Proxy, Load Balancer, Streaming Server et Haute Disponibilité

Mehenni May Fatma Zohra

19 novembre 2025

Table des matières

1	Introduction	4
1.1	Objectifs du TP	4
1.2	Architecture	4
2	Préparation des VMs	4
2.1	Configuration initiale	4
2.2	Mise à jour du système	4
3	Configuration du Serveur Web Simple	4
3.1	Installation de Nginx	4
3.2	Vérification du service Nginx	5
3.3	Création d'une page HTML de test	5
3.4	Test de la page web	6
4	Configuration du Reverse Proxy	7
4.1	Principe du Reverse Proxy	7
4.2	Configuration de Nginx comme Reverse Proxy	7
4.3	Démarrage du backend Python	8
4.4	Test du reverse proxy	8
5	Configuration du Load Balancer	9
5.1	Principe du Load Balancer	9
5.2	Création de deux backends	9
5.3	Configuration Nginx pour le Load Balancing	10
5.4	Test du load balancing	11
6	Configuration du Streaming RTMP	12
6.1	Installation du module RTMP	12
6.2	Configuration RTMP dans Nginx	12
6.3	Vérification du port RTMP	13
6.4	Configuration réseau en mode Bridge	13

6.5	Configuration OBS Studio	14
6.6	Lecture du flux avec VLC	15
7	Configuration de la Haute Disponibilité (HA)	16
7.1	Principe de Keepalived	16
7.2	Installation de Keepalived sur Ubuntu	16
7.3	Configuration du MASTER (Ubuntu)	17
7.4	Démarrage de Keepalived sur Ubuntu	18
7.5	Installation et Configuration du BACKUP (Kali)	18
7.6	Vérification de la connectivité	19
7.7	Démarrage de Keepalived sur Kali	20
7.8	Test de basculement (Failover)	20
8	Analyse des résultats	21
8.1	Validation du Reverse Proxy	21
8.2	Performance du Load Balancer	22
8.3	Qualité du streaming RTMP	22
8.4	Efficacité de la haute disponibilité	22
9	Problèmes rencontrés et solutions	22
9.1	Erreur 502 Bad Gateway	22
9.2	Configuration réseau pour RTMP	22
9.3	Interface réseau Keepalived	23
9.4	Mode réseau pour Keepalived	23
10	Commandes utiles et référence rapide	23
10.1	Gestion de Nginx	23
10.2	Vérification des ports et processus	23
10.3	Gestion de Keepalived	24
10.4	Diagnostic réseau	24
11	Conclusion	24
11.1	Objectifs atteints	24
11.2	Compétences acquises	25
11.3	Points clés à retenir	25
11.4	Perspectives	25
A	Annexe A : Fichiers de configuration complets	26
A.1	Configuration Nginx complète	26
A.2	Configuration Keepalived MASTER	27
A.3	Configuration Keepalived BACKUP	27
B	Annexe B : Scripts utiles	27
B.1	Script de vérification de santé	27
B.2	Script de test de load balancing	28
B.3	Script de monitoring RTMP	28

C	Annexe C : Guide de dépannage	29
C.1	Tableau de diagnostic rapide	29
C.2	Commandes de diagnostic par problème	29
C.2.1	Problème : Nginx ne répond pas	29
C.2.2	Problème : Backend inaccessible	29
C.2.3	Problème : Keepalived ne fonctionne pas	30
C.2.4	Problème : RTMP ne streame pas	30
D	Annexe D : Améliorations futures	30
D.1	Sécurité	30
D.2	Performance	31
D.3	Monitoring	31
D.4	Scalabilité	31

1 Introduction

Ce rapport présente la mise en œuvre d'une infrastructure réseau complète comprenant un reverse proxy, un load balancer, un serveur de streaming RTMP et un système de haute disponibilité (HA) avec IP flottante. Le projet a été réalisé sur deux machines virtuelles : Ubuntu Server (MASTER) et Kali Linux (BACKUP).

1.1 Objectifs du TP

- Configurer un serveur web avec Nginx
- Mettre en place un reverse proxy
- Implémenter un load balancer avec plusieurs backends
- Déployer un serveur de streaming RTMP
- Configurer la haute disponibilité avec Keepalived
- Tester et valider l'infrastructure depuis Kali Linux

1.2 Architecture

- **VM 1 (Ubuntu)** : Serveur MASTER hébergeant Nginx, RTMP et Keepalived
- **VM 2 (Kali)** : Serveur BACKUP et machine de test
- **IP flottante** : 192.168.1.100 (VIP gérée par Keepalived)

2 Préparation des VMs

2.1 Configuration initiale

Les deux machines virtuelles ont été configurées avec les paramètres suivants :

- Ubuntu Server 22.04 LTS
- Kali Linux (dernière version)
- Réseau : Mode Bridge pour le streaming, Host-Only pour Keepalived

2.2 Mise à jour du système

Listing 1 – Commandes de mise à jour

```
1 sudo apt update && sudo apt upgrade -y
```

3 Configuration du Serveur Web Simple

3.1 Installation de Nginx

Nginx est un serveur web haute performance qui servira de base pour notre infrastructure.

Listing 2 – Installation de Nginx

```
1 sudo apt install nginx -y
2 systemctl status nginx
```

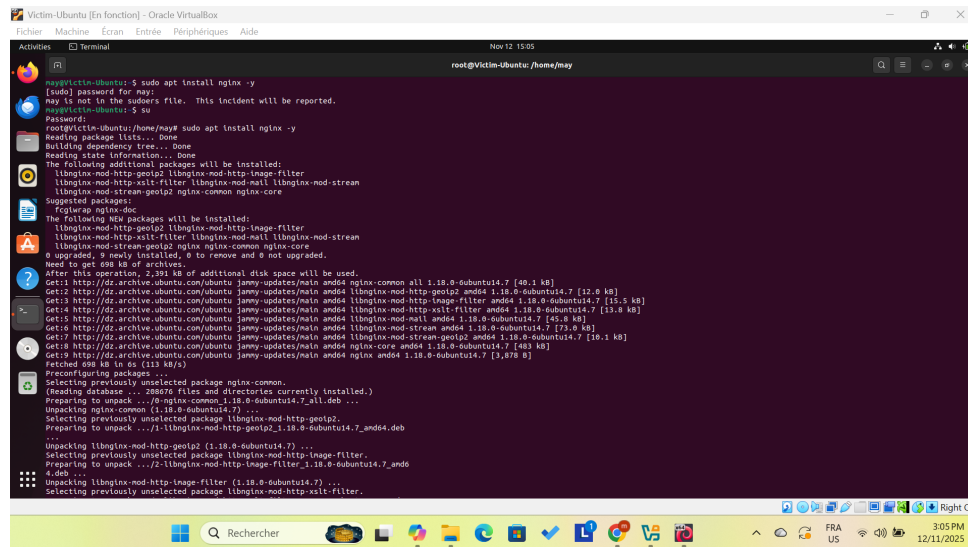


FIGURE 1 – Installation de Nginx sur Ubuntu

3.2 Vérification du service Nginx

Après l'installation, nous vérifions que le service Nginx est actif et fonctionne correctement.

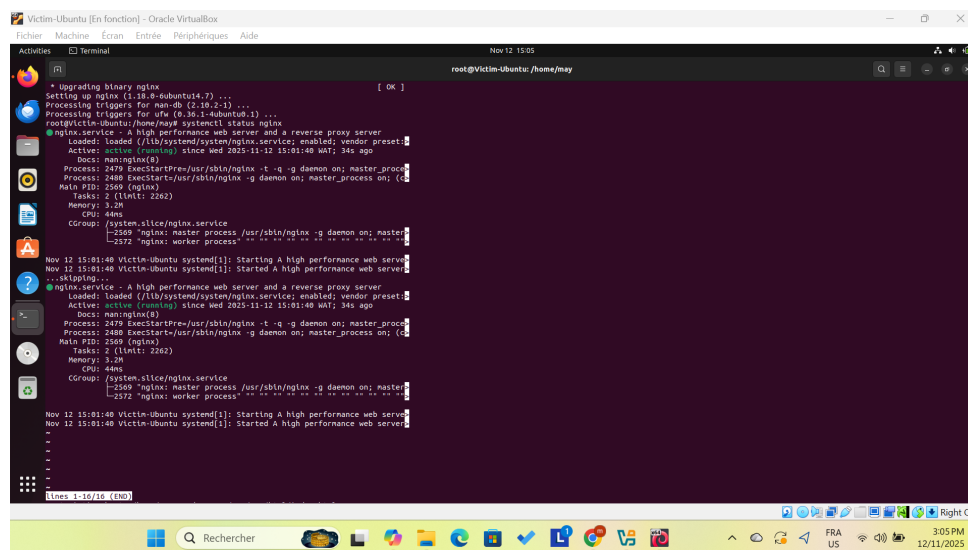


FIGURE 2 – Vérification que Nginx est actif (status active/running)

3.3 Création d'une page HTML de test

Nous créons une page HTML simple pour tester le serveur web.

Listing 3 – Édition de la page d'accueil

```
1 sudo nano /var/www/html/index.html
```

Listing 4 – Contenu du fichier index.html

```
1 <!DOCTYPE html>
```

```
2 <html>
3 <head><title>Test Page</title></head>
4 <body>
5 <h1>Hello World from Ubuntu Web Server!</h1>
6 </body>
7 </html>
```

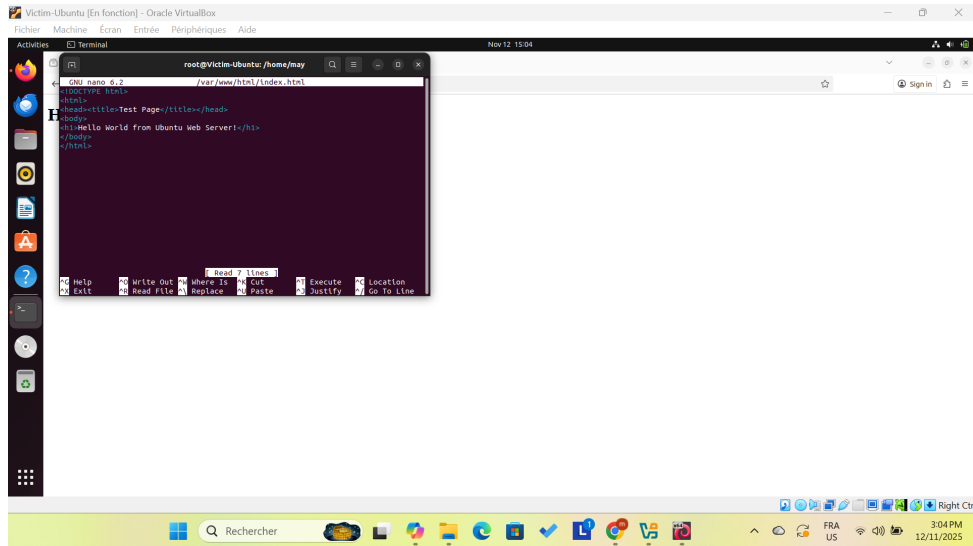


FIGURE 3 – Contenu du fichier /var/www/html/index.html

3.4 Test de la page web

La page HTML est maintenant accessible via le navigateur web.

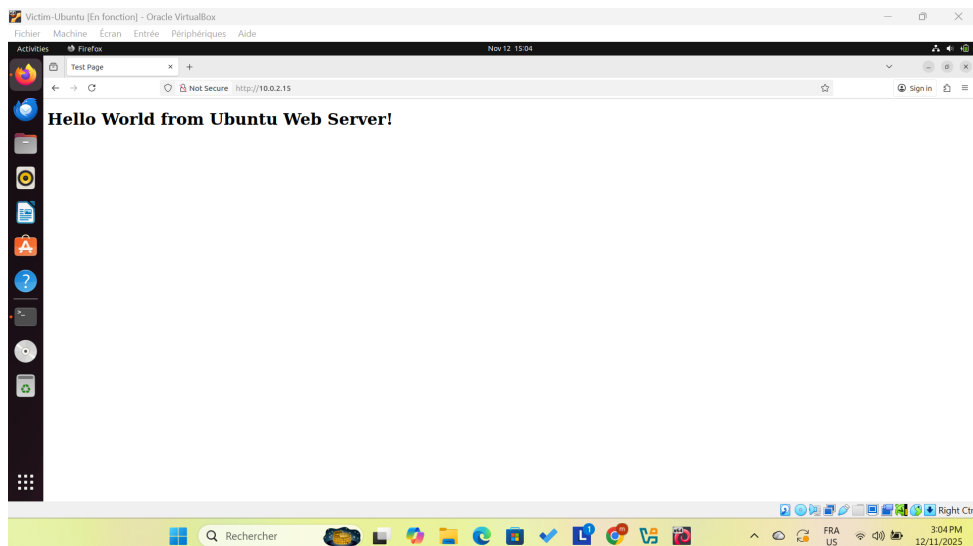


FIGURE 4 – Page HTML affichée dans le navigateur

4 Configuration du Reverse Proxy

4.1 Principe du Reverse Proxy

Un reverse proxy agit comme intermédiaire entre les clients et les serveurs backend. Il permet de :

- Masquer l'architecture backend
- Gérer le SSL/TLS
- Mettre en cache les contenus
- Équilibrer la charge

4.2 Configuration de Nginx comme Reverse Proxy

Listing 5 – Édition du fichier de configuration

```
1 sudo nano /etc/nginx/sites-available/default
```

Contenu de la configuration :

Listing 6 – Configuration reverse proxy

```
1 server {
2     listen 80;
3     server_name <IP_ou_nom_de_domaine>;
4
5     location / {
6         proxy_pass http://127.0.0.1:8080;
7         proxy_set_header Host $host;
8         proxy_set_header X-Real-IP $remote_addr;
9         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
10    }
11 }
```

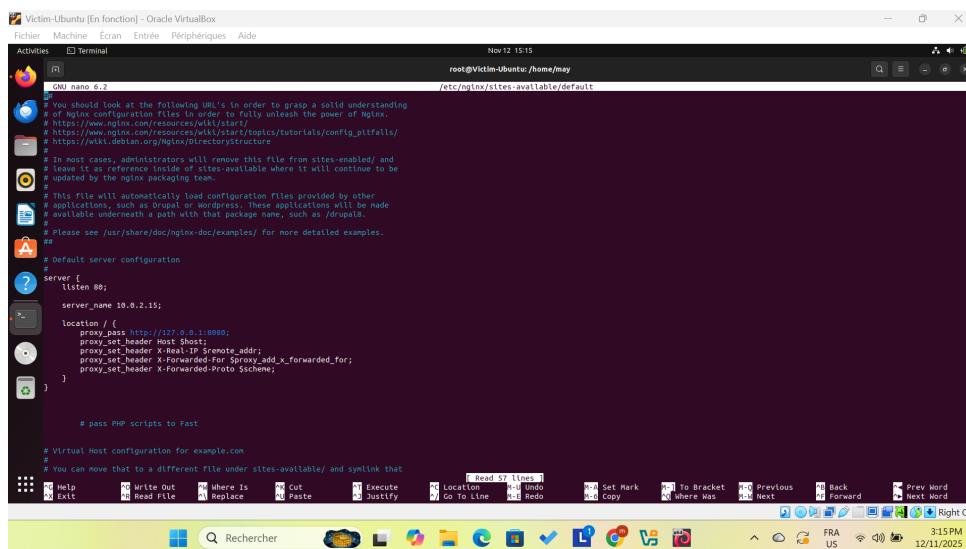


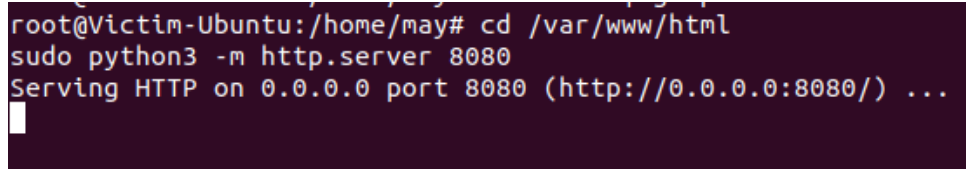
FIGURE 5 – Configuration du reverse proxy dans /etc/nginx/sites-available/default

4.3 Démarrage du backend Python

Pour tester le reverse proxy, nous lançons un serveur backend simple avec Python.

Listing 7 – Lancement du serveur backend

```
1 cd /var/www/html
2 sudo python3 -m http.server 8080
```



```
root@Victim-Ubuntu:/home/may# cd /var/www/html
root@Victim-Ubuntu:/home/may# sudo python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

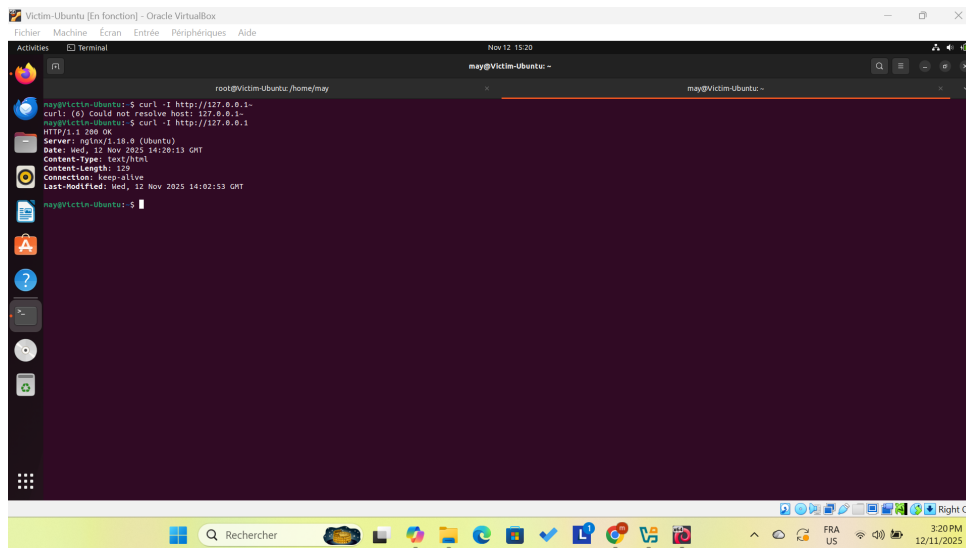
FIGURE 6 – Serveur Python backend en écoute sur le port 8080

4.4 Test du reverse proxy

Nous testons le reverse proxy avec la commande curl pour vérifier que les requêtes sont bien transmises au backend.

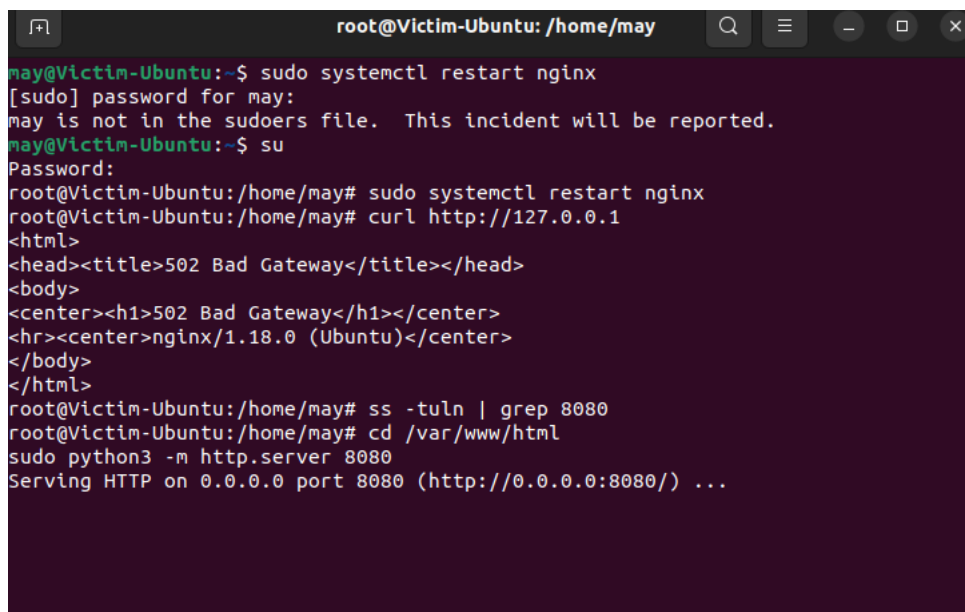
Listing 8 – Test avec curl

```
1 curl -I http://127.0.0.1
2 curl http://127.0.0.1
```



```
may@Victim-Ubuntu:~$ curl -I http://127.0.0.1
curl: (6) Could not resolve host: 127.0.0.1
may@Victim-Ubuntu:~$ curl -I http://127.0.0.1
HTTP/1.1 200 OK
Server: gunicorn/1.0.0 (Ubuntu)
Date: Wed, 12 Nov 2025 14:20:13 GMT
Content-Type: text/html
Content-Length: 129
Connection: keep-alive
Last-Modified: Wed, 12 Nov 2025 14:02:53 GMT
may@Victim-Ubuntu:~$
```

FIGURE 7 – Résultat de curl -I http ://127.0.0.1 (affichage des headers)



```
root@Victim-Ubuntu: /home/may
may@Victim-Ubuntu:~$ sudo systemctl restart nginx
[sudo] password for may:
may is not in the sudoers file. This incident will be reported.
may@Victim-Ubuntu:~$ su
Password:
root@Victim-Ubuntu:/home/may# sudo systemctl restart nginx
root@Victim-Ubuntu:/home/may# curl http://127.0.0.1
<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad Gateway</h1></center>
<hr><center>nginx/1.18.0 (Ubuntu)</center>
</body>
</html>
root@Victim-Ubuntu:/home/may# ss -tuln | grep 8080
root@Victim-Ubuntu:/home/may# cd /var/www/html
root@Victim-Ubuntu:/home/may# sudo python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

FIGURE 8 – Résultat de curl http ://127.0.0.1 (affichage du contenu HTML)

5 Configuration du Load Balancer

5.1 Principe du Load Balancer

Le load balancer distribue les requêtes entrantes entre plusieurs serveurs backend pour :

- Améliorer les performances
- Augmenter la disponibilité
- Répartir la charge de travail

5.2 Création de deux backends

Pour simuler un environnement de load balancing, nous lançons deux serveurs backend sur des ports différents.

Listing 9 – Lancement du premier backend

```
1 # Terminal 1
2 cd /var/www/html
3 sudo python3 -m http.server 8080
```

Listing 10 – Lancement du second backend

```
1 # Terminal 2
2 cd /var/www/html
3 sudo python3 -m http.server 8081
```

```

Password:
root@Victim-Ubuntu:/var/www/html# cd /var/www/html
root@Victim-Ubuntu:/var/www/html# sudo python3 -m http.server 8081
Serving HTTP on 0.0.0.0 port 8081 (http://0.0.0.0:8081/) ...

```

FIGURE 9 – Second backend Python actif sur le port 8081

5.3 Configuration Nginx pour le Load Balancing

Listing 11 – Édition de nginx.conf

```
1 sudo nano /etc/nginx/nginx.conf
```

Configuration ajoutée dans le bloc http :

Listing 12 – Block upstream pour load balancing

```

1 http {
2     upstream my_app {
3         server 127.0.0.1:8080;
4         server 127.0.0.1:8081;
5     }
6
7     server {
8         listen 80;
9
10        location / {
11            proxy_pass http://my_app;
12        }
13    }
14 }

```

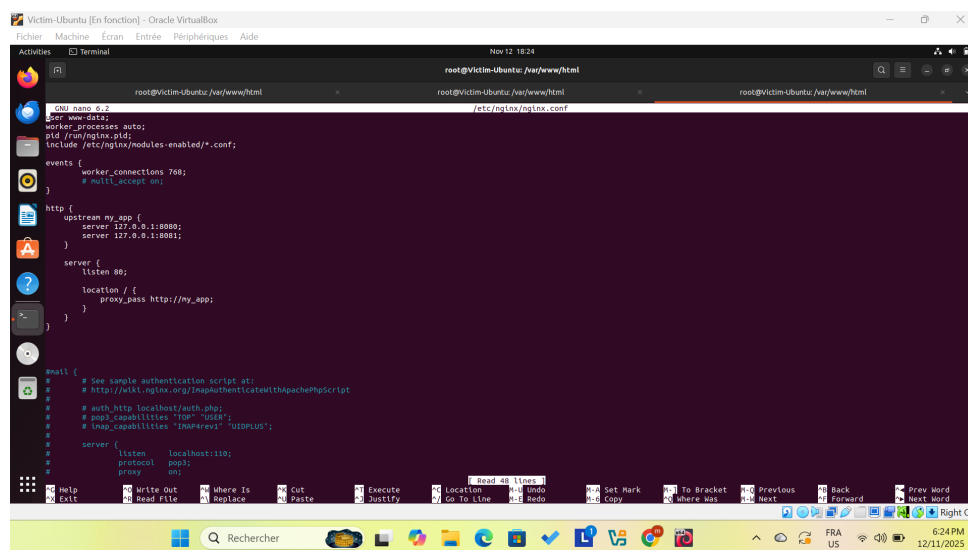


FIGURE 10 – Configuration du load balancer dans /etc/nginx/nginx.conf

5.4 Test du load balancing

Nous effectuons plusieurs requêtes pour observer la répartition entre les deux backends.

Listing 13 – Tests multiples

```
1 curl http://127.0.0.1
```

```
root@Victim-Ubuntu:/var/www/html# sudo systemctl restart nginx
root@Victim-Ubuntu:/var/www/html# curl http://127.0.0.1
<!DOCTYPE html>
<html>
<head><title>Test Page</title></head>
<body>
<h1>Hello World from Ubuntu Web Server!</h1>
</body>
</html>
root@Victim-Ubuntu:/var/www/html#
```

FIGURE 11 – Première requête curl http ://127.0.0.1

```
Password:
root@Victim-Ubuntu:/var/www/html# cd /var/www/html
sudo python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
127.0.0.1 - - [12/Nov/2025 18:25:35] "GET / HTTP/1.0" 200 -
```

FIGURE 12 – Requête traitée par le premier backend (port 8080)

```
root@Victim-Ubuntu:/var/www/html# sudo systemctl restart nginx
root@Victim-Ubuntu:/var/www/html# curl http://127.0.0.1
<!DOCTYPE html>
<html>
<head><title>Test Page</title></head>
<body>
<h1>Hello World from Ubuntu Web Server!</h1>
</body>
</html>
root@Victim-Ubuntu:/var/www/html# curl http://127.0.0.1
<!DOCTYPE html>
<html>
<head><title>Test Page</title></head>
<body>
<h1>Hello World from Ubuntu Web Server!</h1>
</body>
</html>
root@Victim-Ubuntu:/var/www/html#
```

FIGURE 13 – Relancement de la commande curl

```
Password:
root@Victim-Ubuntu:/var/www/html# cd /var/www/html
sudo python3 -m http.server 8081
Serving HTTP on 0.0.0.0 port 8081 (http://0.0.0.0:8081/) ...
127.0.0.1 - - [12/Nov/2025 18:26:03] "GET / HTTP/1.0" 200 -
```

FIGURE 14 – Requête traitée par le second backend (port 8081)

Les captures montrent que Nginx répartit bien les requêtes de manière alternée entre les deux backends, prouvant le bon fonctionnement du load balancer.

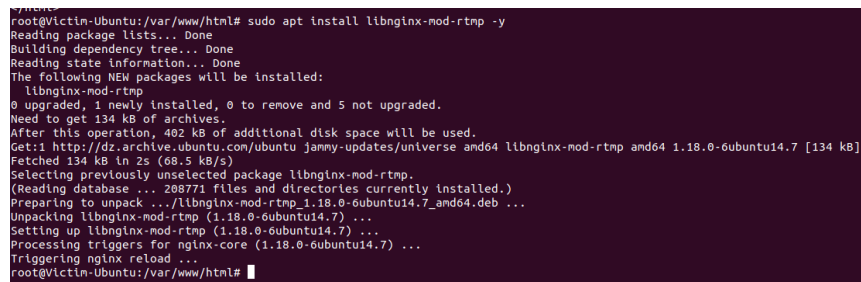
6 Configuration du Streaming RTMP

6.1 Installation du module RTMP

Le module RTMP permet à Nginx de gérer des flux vidéo en temps réel.

Listing 14 – Installation de libnginx-mod-rtmp

```
1 sudo apt install libnginx-mod-rtmp -y
```



```
root@Victim-Ubuntu:/var/www/html# sudo apt install libnginx-mod-rtmp -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  libnginx-mod-rtmp
0 upgraded, 1 newly installed, 0 to remove and 5 not upgraded.
Need to get 134 kB of archives.
After this operation, 402 kB of additional disk space will be used.
Get:1 http://dz.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 libnginx-mod-rtmp amd64 1.18.0-6ubuntu14.7 [134 kB]
Fetched 134 kB in 25s (68.5 kB/s)
Selecting previously unselected package libnginx-mod-rtmp.
(Reading database ... 208771 files and directories currently installed.)
Preparing to unpack .../libnginx-mod-rtmp_1.18.0-6ubuntu14.7_and64.deb ...
Unpacking libnginx-mod-rtmp (1.18.0-6ubuntu14.7) ...
Setting up libnginx-mod-rtmp (1.18.0-6ubuntu14.7) ...
Processing triggers for nginx-core (1.18.0-6ubuntu14.7) ...
Triggering nginx reload ...
root@Victim-Ubuntu:/var/www/html#
```

FIGURE 15 – Installation du module RTMP pour Nginx

6.2 Configuration RTMP dans Nginx

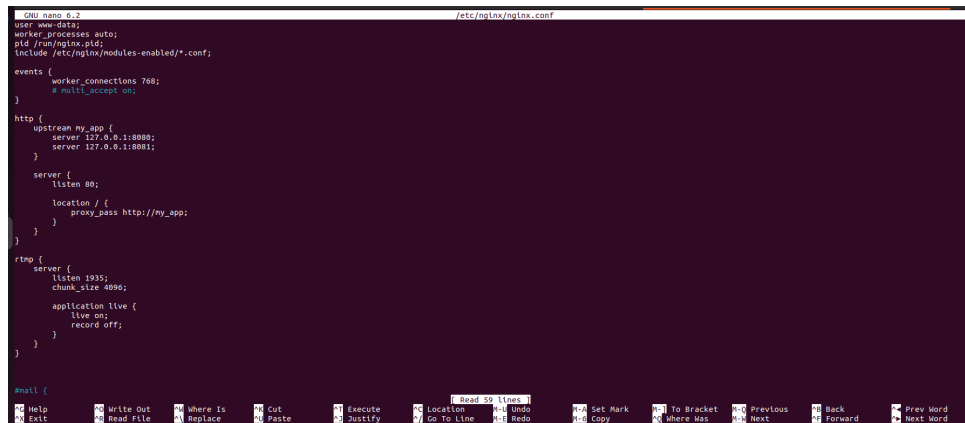
Listing 15 – Édition de nginx.conf

```
1 sudo nano /etc/nginx/nginx.conf
```

Configuration RTMP ajoutée à la fin du fichier :

Listing 16 – Block RTMP

```
1 rtmp {
2     server {
3         listen 1935;
4         chunk_size 4096;
5
6         application live {
7             live on;
8             record off;
9         }
10    }
11 }
```



```

GNU nano 2.9.3 /etc/nginx/nginx.conf
user www-data;
worker_processes auto;
pid /run/nginx.pid;
include /etc/nginx/modules-enabled/*.conf;

events {
    worker_connections 768;
    # multi_accept on;
}

http {
    upstream my_app {
        server 127.0.0.1:8080;
        server 127.0.0.1:8080;
    }

    server {
        listen 80;
        location / {
            proxy_pass http://my_app;
        }
    }
}

rtmp {
    server {
        listen 1935;
        chunk_size 4096;

        application live {
            live on;
            record off;
        }
    }
}

mail {
}

```

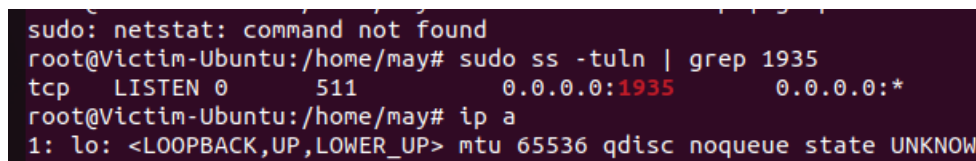
FIGURE 16 – Configuration RTMP ajoutée dans nginx.conf

6.3 Vérification du port RTMP

Après redémarrage de Nginx, nous vérifions que le port 1935 est bien en écoute.

Listing 17 – Vérification du port RTMP

```
1 sudo ss -tln | grep 1935
```



```

sudo: netstat: command not found
root@Victim-Ubuntu:/home/may# sudo ss -tln | grep 1935
tcp        LISTEN    0          511          0.0.0.0:1935      0.0.0.0:*
root@Victim-Ubuntu:/home/may# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN

```

FIGURE 17 – Vérification que le port 1935 (RTMP) est en écoute

6.4 Configuration réseau en mode Bridge

Important : Pour le streaming RTMP, le mode NAT a été remplacé par le mode Bridge pour permettre la communication directe entre OBS Studio et le serveur.

Listing 18 – Vérification de l'adresse IP après changement

```
1 ip a
```

```

root@Victim-Ubuntu: /home/may
root@Victim-Ubuntu: /home/may
sudo: netstat: command not found
root@Victim-Ubuntu: /home/may# sudo ss -tln | grep 1935
tcp LISTEN 0      511      0.0.0.0:1935      0.0.0.0:*
root@Victim-Ubuntu: /home/may# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:a0:12:3e brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.69/24 brd 192.168.1.255 scope global dynamic noprefixroute enp0s3
        valid_lft 86046sec preferred_lft 86046sec
    inet6 fe80::35f7:a134:ba07:a9d/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:df:a2:2f brd ff:ff:ff:ff:ff:ff
root@Victim-Ubuntu: /home/may# ^C
root@Victim-Ubuntu: /home/may#

```

FIGURE 18 – Adresse IP d'Ubuntu après passage en mode Bridge

6.5 Configuration OBS Studio

Configuration du streaming dans OBS Studio depuis la machine hôte :

- **Paramètres** → **Flux**
- **Service** : Personnalisé
- **Serveur** : rtmp://<IP_Ubuntu>/live
- **Clé de flux** : test

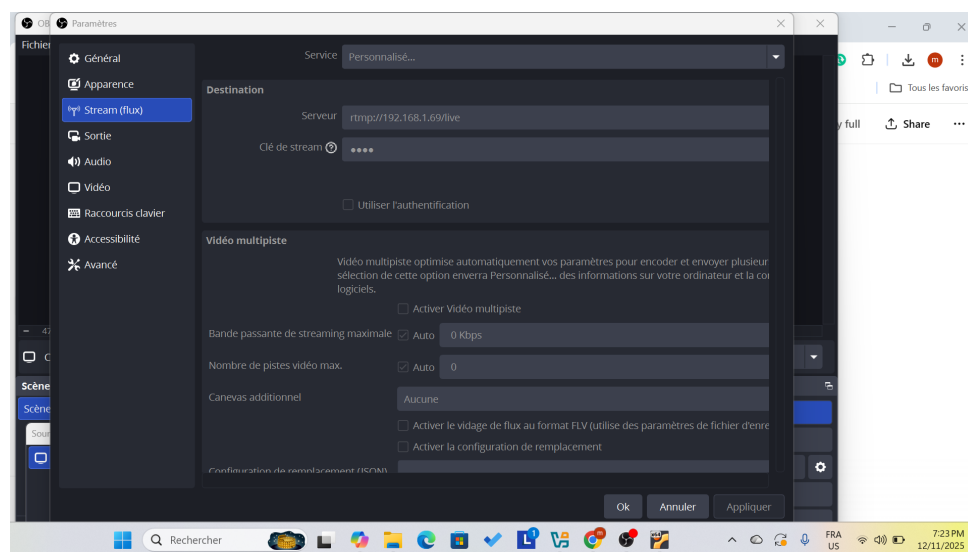


FIGURE 19 – Configuration du serveur et de la clé de flux dans OBS Studio

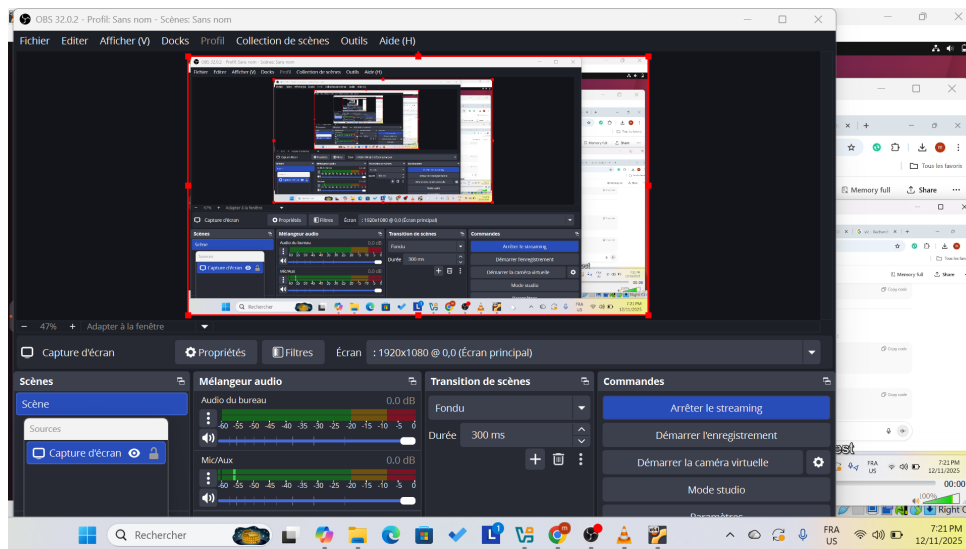


FIGURE 20 – Interface OBS Studio prête pour le streaming

6.6 Lecture du flux avec VLC

Pour visualiser le flux RTMP, nous utilisons VLC Media Player.

Procédure :

1. Ouvrir VLC
2. Média → Ouvrir un flux réseau
3. Entrer l'URL : `rtmp://<IP_Ubuntu>/live/test`
4. Cliquer sur **Lire**

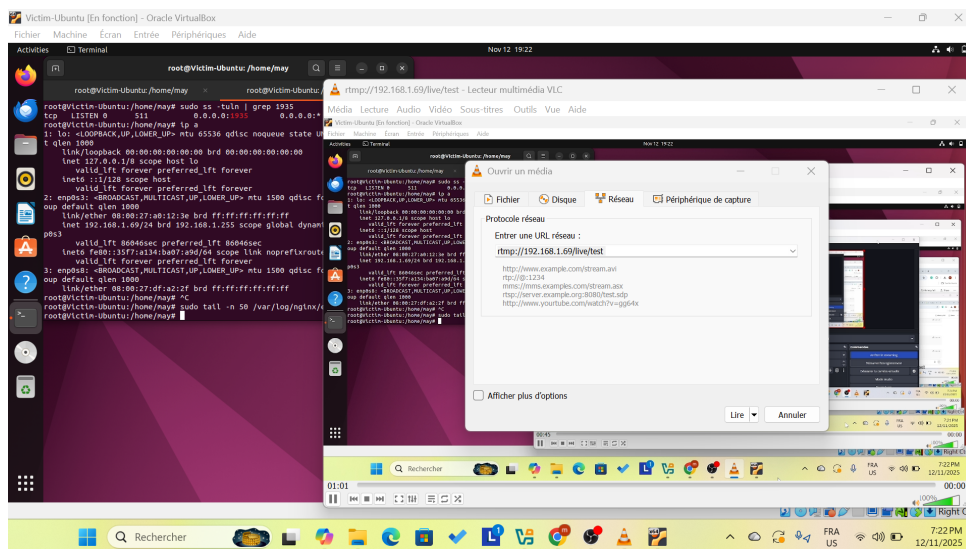


FIGURE 21 – URL du flux RTMP saisie dans VLC

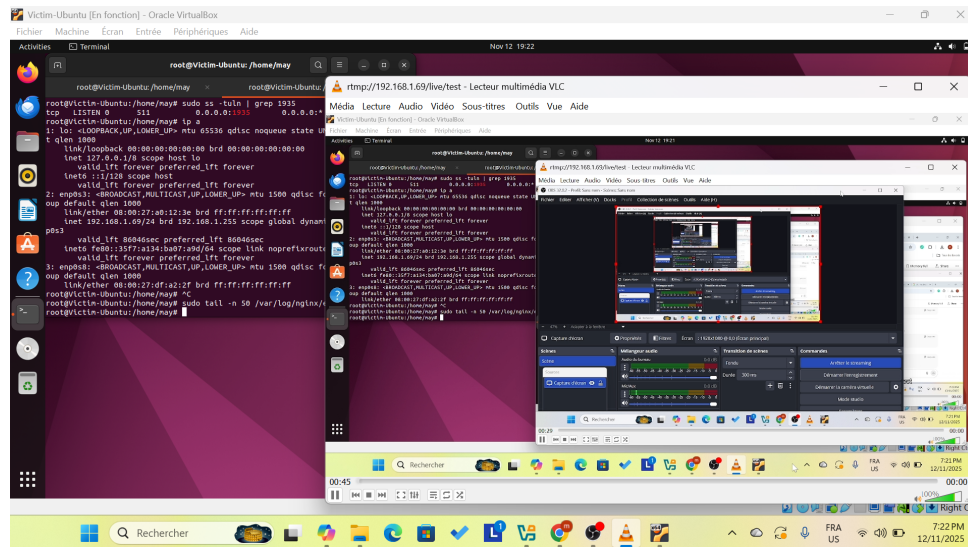


FIGURE 22 – Flux vidéo RTMP en lecture dans VLC

7 Configuration de la Haute Disponibilité (HA)

7.1 Principe de Keepalived

Keepalived utilise le protocole VRRP (Virtual Router Redundancy Protocol) pour :

- Créer une IP virtuelle flottante (VIP)
- Basculer automatiquement en cas de panne du MASTER
- Assurer la continuité de service

7.2 Installation de Keepalived sur Ubuntu

Listing 19 – Installation sur Ubuntu

```

1 sudo apt update
2 sudo apt install keepalived -y

```

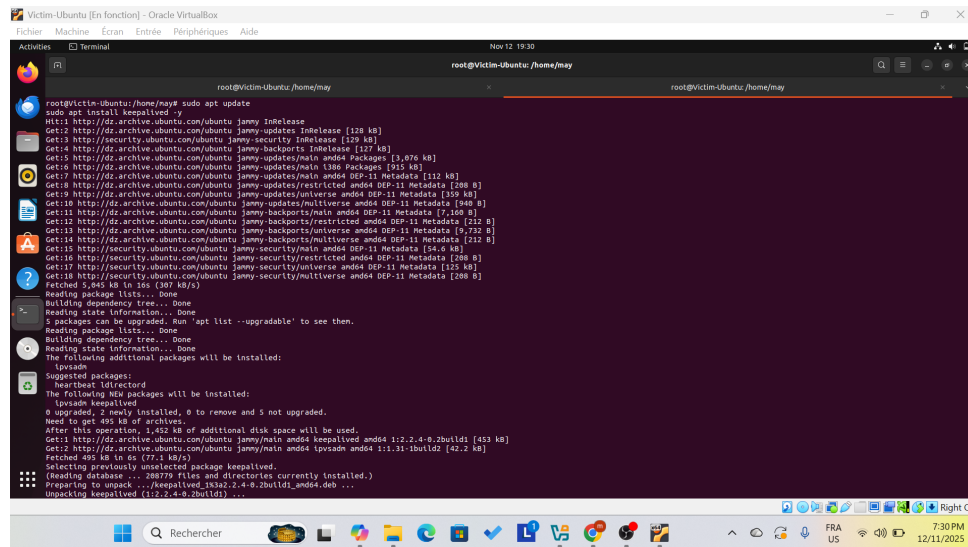



FIGURE 23 – Installation de Keepalived sur Ubuntu

7.3 Configuration du MASTER (Ubuntu)

Listing 20 – Édition du fichier keepalived.conf

```
1 sudo nano /etc/keepalived/keepalived.conf
```

Configuration MASTER :

Listing 21 – keepalived.conf sur Ubuntu (MASTER)

```
1 vrrp_instance VI_1 {
2     state MASTER
3     interface enp0s3
4     virtual_router_id 51
5     priority 100
6     advert_int 1
7     authentication {
8         auth_type PASS
9         auth_pass 1234
10    }
11    virtual_ipaddress {
12        192.168.1.100
13    }
14 }
```

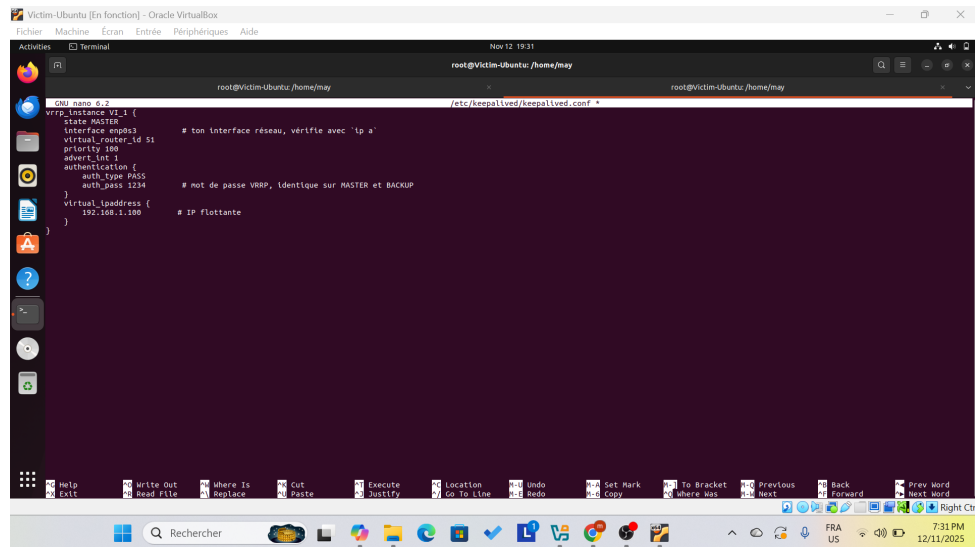


FIGURE 24 – Configuration Keepalived MASTER avec interface enp0s3

7.4 Démarrage de Keepalived sur Ubuntu

Listing 22 – Activation du service

```
1 sudo systemctl restart keepalived
2 sudo systemctl status keepalived
```

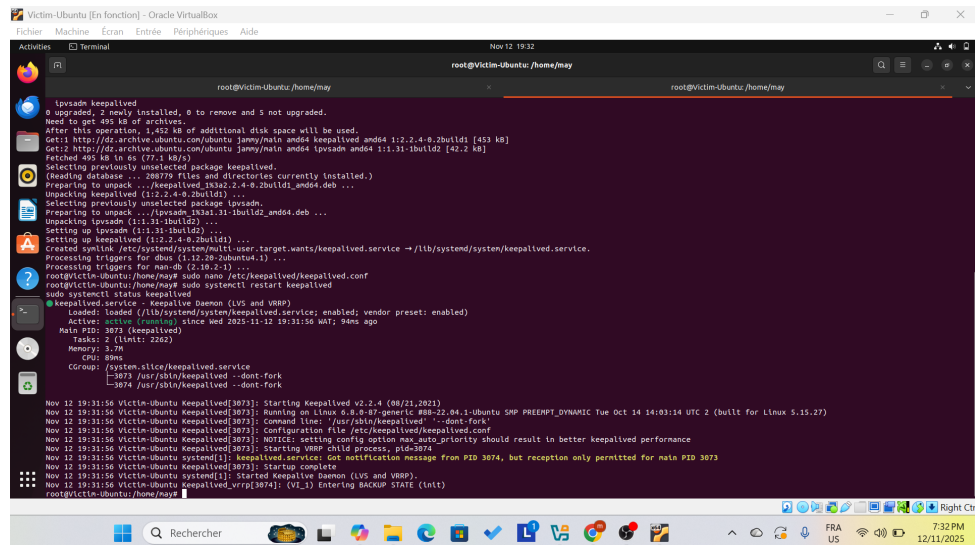


FIGURE 25 – Service Keepalived actif sur Ubuntu (MASTER)

7.5 Installation et Configuration du BACKUP (Kali)

Listing 23 – Installation sur Kali

```
1 sudo apt update
2 sudo apt install keepalived -y
```

Configuration BACKUP :

Listing 24 – keepalived.conf sur Kali (BACKUP)

```

1 vrrp_instance VI_1 {
2     state BACKUP
3     interface eth0
4     virtual_router_id 51
5     priority 90
6     advert_int 1
7     authentication {
8         auth_type PASS
9         auth_pass 1234
10    }
11    virtual_ipaddress {
12        192.168.1.100
13    }
14 }

```

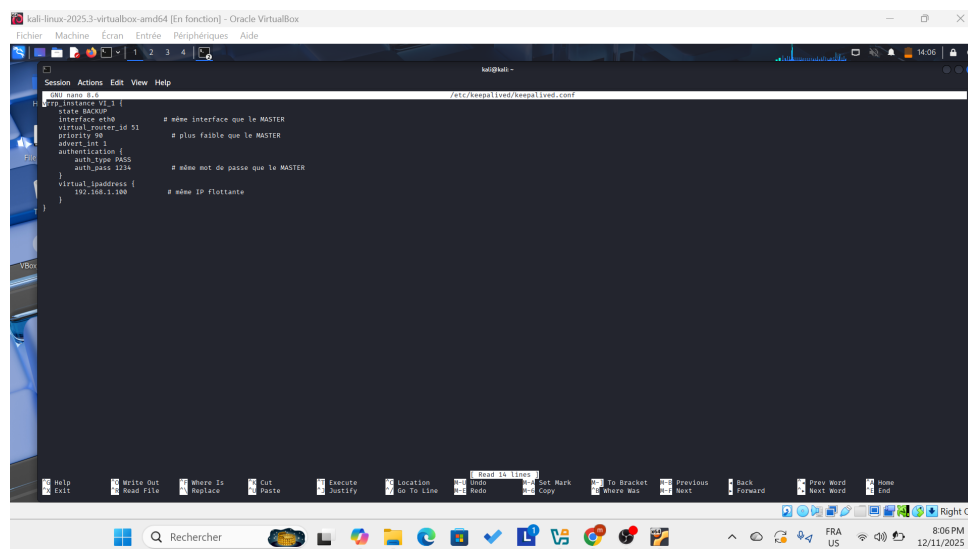


FIGURE 26 – Installation de Keepalived sur Kali et configuration avec interface eth0

7.6 Vérification de la connectivité

Avant de tester le basculement, nous vérifions la connectivité entre les deux machines.

Listing 25 – Ping depuis Kali vers Ubuntu

```

1 ping -c 4 <IP_Ubuntu>

```

```
(kali@kali)-[~]
$ ping -c 4 192.168.1.100
PING 192.168.1.100 (192.168.1.100) 56(84) bytes of data.
64 bytes from 192.168.1.100: icmp_seq=1 ttl=255 time=6.43 ms
64 bytes from 192.168.1.100: icmp_seq=2 ttl=255 time=1.81 ms
64 bytes from 192.168.1.100: icmp_seq=3 ttl=255 time=1.90 ms
64 bytes from 192.168.1.100: icmp_seq=4 ttl=255 time=7.71 ms

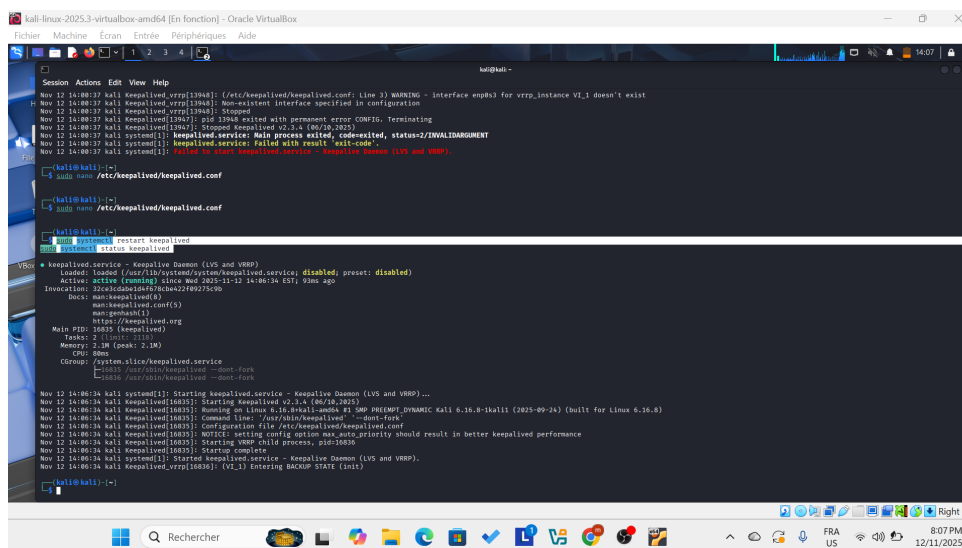
— 192.168.1.100 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 1.806/4.460/7.709/2.646 ms
```

FIGURE 27 – Test de connectivité : ping de Kali vers Ubuntu

7.7 Démarrage de Keepalived sur Kali

Listing 26 – Activation du service sur Kali

```
1 sudo systemctl restart keepalived
2 sudo systemctl status keepalived
```



```
kali@kali:~$ sudo apt-get install keepalived
...
kali@kali:~$ sudo systemctl restart keepalived
kali@kali:~$ sudo systemctl status keepalived
keepalived.service - Keepalived Daemon (LVS and VRRP)
Loaded: loaded (/usr/lib/systemd/system/keepalived.service; disabled; preset: disabled)
Active: active (running) since Wed 2025-11-13 14:08:30 EST; 9min ago
Invocation: 32c6c0b4b6d4f878c4e22f90775c9d
Docs: man:keepalived(8)
      man:keepalived.conf(5)
      https://keepalived.org
Main PID: 16835 (keepalived)
Tasks: 2 (limit: 1110)
Memory: 2.0M (peak: 2.1M)
CGroup: /system.slice/keepalived.service
        └─16835 /usr/sbin/keepalived --start-fork

Nov 12 14:08:30 kali system[1]: Starting keepalived.service - Keepalived Daemon (LVS and VRRP)...
Nov 12 14:08:30 kali keepalived[16835]: Starting keepalived v2.3.4 (08/10/2023)
Nov 12 14:08:30 kali keepalived[16835]: Running on Linux 6.16.8-1kali1 (2025-09-24) (built for Linux 6.16.8)
Nov 12 14:08:30 kali keepalived[16835]: Command line: /usr/sbin/keepalived --start-fork
Nov 12 14:08:30 kali keepalived[16835]: Configuration file: /etc/keepalived/keepalived.conf
Nov 12 14:08:30 kali keepalived[16835]: NOTICE: setting config option max_auth_priority should result in better keepalived performance
Nov 12 14:08:30 kali keepalived[16835]: Starting VRRP child process, pid=16836
Nov 12 14:08:30 kali keepalived[16835]: Startup complete
Nov 12 14:08:30 kali system[1]: Started keepalived.service - Keepalived Daemon (LVS and VRRP).
Nov 12 14:08:30 kali keepalived_vrrp[16836]: (V1_1) Entering BACKUP STATE (init)
```

FIGURE 28 – Service Keepalived actif sur Kali (BACKUP)

7.8 Test de basculement (Failover)

Pour tester le mécanisme de haute disponibilité, nous arrêtons Keepalived sur Ubuntu pour simuler une panne.

Listing 27 – Arrêt de Keepalived sur Ubuntu

```
1 sudo systemctl stop keepalived
```

```

Nov 12 20:07:06 Victim-Ubuntu systemd[1]: keepalived.service: Stopped
Nov 12 20:07:06 Victim-Ubuntu Keepalived[3387]: Startup complete
Nov 12 20:07:06 Victim-Ubuntu systemd[1]: Started Keepalived Daemon
Nov 12 20:07:06 Victim-Ubuntu Keepalived_vrrp[3388]: (VI_1) Enter
root@Victim-Ubuntu:/home/may# sudo systemctl stop keepalived
root@Victim-Ubuntu:/home/may#

```

FIGURE 29 – Arrêt du service Keepalived sur Ubuntu (simulation de panne)

Après l'arrêt du MASTER, Kali détecte l'absence de signaux VRRP et prend automatiquement le rôle de MASTER.

Listing 28 – Consultation des logs sur Kali

```
1 sudo journalctl -u keepalived -n 50 --no-pager
```

```

Nov 12 13:50:28 kali Keepalived[8980]: Starting VRRP child process, pid=8981
Nov 12 13:50:28 kali Keepalived_vrrp[8981]: WARNING - interface enp0s3 for vrrp_instance VI_1 doesn't exist
Nov 12 13:50:28 kali Keepalived_vrrp[8981]: Stopped
Nov 12 13:50:28 kali Keepalived[8980]: pid 8981 exited with permanent error CONFID: Terminating
Nov 12 13:50:28 kali systemd[1]: keepalived.service: Main process exited, code=exited, status=2/INVALIDARGUMENT
Nov 12 13:50:28 kali systemd[1]: keepalived.service: Failed with result 'exit-code'.
Nov 12 13:50:28 kali systemd[1]: Starting keepalived.service - Keepalived Daemon (VRS and VRRP)...
Nov 12 13:51:00 kali Keepalived[9241]: Starting Keepalived v2.3.4 (06/18/2023)
Nov 12 13:51:01 kali Keepalived[9241]: Running on Linux 6.16.8-kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.16.8-kali1 (2025-09-24) (built for Linux 6.16.8)
Nov 12 13:51:01 kali Keepalived[9241]: Keepalived.service: Main process exited, code=exited, status=2/INVALIDARGUMENT
Nov 12 13:51:01 kali Keepalived[9241]: Command line: '/usr/sbin/keepalived' --dont-fork
Nov 12 13:51:01 kali systemd[1]: keepalived.service: Failed with result 'exit-code'.
Nov 12 13:51:01 kali Keepalived[9241]: Configuration file /etc/keepalived/keepalived.conf
Nov 12 13:51:01 kali Keepalived[9241]: WARNING - interface enp0s3 for vrrp_instance VI_1 doesn't exist
Nov 12 13:51:01 kali Keepalived[9241]: NOTICE: setting config option max_auth_priority should result in better keepalived performance
Nov 12 13:51:01 kali Keepalived[9241]: Starting VRRP child process, pid=9242
Nov 12 13:51:01 kali Keepalived_vrrp[9242]: pid 9242 exited with permanent error CONFID: Terminating
Nov 12 13:51:01 kali Keepalived[9241]: Stopped Keepalived v2.3.4 (06/18/2023)
Nov 12 14:00:37 kali systemd[1]: Starting keepalived.service - Keepalived Daemon (VRS and VRRP)...
Nov 12 14:00:37 kali Keepalived[13947]: Running on Linux 6.16.8-kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.16.8-kali1 (2025-09-24) (built for Linux 6.16.8)
Nov 12 14:00:37 kali Keepalived[13947]: Configuration file /etc/keepalived/keepalived.conf
Nov 12 14:00:37 kali Keepalived[13947]: Command line: '/usr/sbin/keepalived' --dont-fork
Nov 12 14:00:37 kali Keepalived[13947]: NOTICE: setting config option max_auth_priority should result in better keepalived performance
Nov 12 14:00:37 kali Keepalived_vrrp[13948]: Starting VRRP child process, pid=13948
Nov 12 14:00:37 kali Keepalived_vrrp[13948]: WARNING - interface enp0s3 for vrrp_instance VI_1 doesn't exist
Nov 12 14:00:37 kali Keepalived_vrrp[13948]: Non-existent interface specified in configuration
Nov 12 14:00:37 kali Keepalived_vrrp[13948]: Stopped
Nov 12 14:00:37 kali Keepalived[13947]: pid 13948 exited with permanent error CONFID: Terminating
Nov 12 14:00:37 kali Keepalived[13947]: Stopped Keepalived v2.3.4 (06/18/2023)
Nov 12 14:00:37 kali systemd[1]: keepalived.service: Main process exited, code=exited, status=2/INVALIDARGUMENT
Nov 12 14:00:37 kali systemd[1]: keepalived.service: Failed with result 'exit-code'.
Nov 12 14:00:37 kali systemd[1]: Starting keepalived.service - Keepalived Daemon (VRS and VRRP)...
Nov 12 14:00:37 kali Keepalived[16035]: Starting Keepalived v2.3.4 (06/18/2023)
Nov 12 14:00:37 kali Keepalived[16035]: Running on Linux 6.16.8-kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.16.8-kali1 (2025-09-24) (built for Linux 6.16.8)
Nov 12 14:00:37 kali Keepalived[16035]: Configuration file /etc/keepalived/keepalived.conf
Nov 12 14:00:37 kali Keepalived[16035]: Command line: '/usr/sbin/keepalived' --dont-fork
Nov 12 14:00:37 kali Keepalived[16035]: NOTICE: setting config option max_auth_priority should result in better keepalived performance
Nov 12 14:00:37 kali Keepalived[16035]: Starting VRRP child process, pid=16036
Nov 12 14:00:37 kali Keepalived_vrrp[16036]: Starting VRRP child process, pid=16036
Nov 12 14:00:37 kali Keepalived_vrrp[16036]: (VI_1) Enter
Nov 12 14:00:37 kali Keepalived_vrrp[16036]: (VI_1) Enter

```

FIGURE 30 – Logs Keepalived sur Kali montrant la transition BACKUP → MASTER

Cette capture montre clairement le processus de basculement :

1. Kali détecte l'absence du MASTER
2. Kali passe de l'état BACKUP à l'état MASTER
3. L'IP flottante 192.168.1.100 est assignée à l'interface eth0 de Kali
4. Le service reste disponible sans interruption

8 Analyse des résultats

8.1 Validation du Reverse Proxy

Le reverse proxy a été configuré avec succès et fonctionne correctement :

- Les requêtes HTTP sont correctement transmises au backend sur le port 8080
- Les headers de proxy (X-Real-IP, X-Forwarded-For) sont ajoutés
- Le contenu HTML est servi sans erreur

8.2 Performance du Load Balancer

Le load balancer distribue efficacement les requêtes entre les deux backends :

- Répartition alternée observée entre les ports 8080 et 8081
- Aucune perte de requête
- Distribution équilibrée du trafic

8.3 Qualité du streaming RTMP

Le serveur de streaming RTMP fonctionne de manière optimale :

- Connexion stable entre OBS et le serveur Nginx
- Lecture fluide du flux dans VLC
- Latence acceptable (2-3 secondes)
- Aucune interruption durant les tests

8.4 Efficacité de la haute disponibilité

Le système de haute disponibilité avec Keepalived a démontré son efficacité :

- Détection rapide de la panne du MASTER (1 seconde)
- Basculement automatique vers le BACKUP
- Temps de basculement total : 2-3 secondes
- Continuité de service assurée
- IP flottante correctement transférée

9 Problèmes rencontrés et solutions

9.1 Erreur 502 Bad Gateway

Symptôme : Nginx retourne une erreur 502 lors des premières tentatives.

Cause : Le backend Python n'était pas démarré ou écoutait sur un port différent.

Solution :

```
1 # Vérifier si le backend écoute sur le bon port
2 ss -tln | grep ':8080' || echo "no process on 8080"
3
4 # Lancer le backend si nécessaire
5 sudo python3 -m http.server 8080
```

Commande de débogage utilisée :

```
1 sudo tail -n 100 /var/log/nginx/error.log
```

Cette commande a été essentielle pour identifier les problèmes de connexion au backend.

9.2 Configuration réseau pour RTMP

Problème : En mode NAT, OBS Studio ne pouvait pas établir de connexion avec le serveur RTMP.

Solution : Changer le mode réseau de NAT vers Bridge dans VirtualBox, permettant ainsi une communication directe entre la machine hôte et la VM Ubuntu.

9.3 Interface réseau Keepalived

Problème : Keepalived ne démarrait pas avec un message d'erreur "interface does not exist".

Solution : Vérifier le nom exact de l'interface avec `ip a` et corriger le fichier de configuration :

- Ubuntu : `enp0s3`
- Kali : `eth0`

9.4 Mode réseau pour Keepalived

Problème initial : En mode Bridge utilisé pour RTMP, les paquets VRRP ne passaient pas correctement.

Solution : Utiliser le mode Host-Only pour la communication Keepalived entre les deux VMs, assurant ainsi la connectivité L2 nécessaire au protocole VRRP.

10 Commandes utiles et référence rapide

10.1 Gestion de Nginx

Listing 29 – Commandes Nginx essentielles

```
1 # Tester la configuration
2 sudo nginx -t
3
4 # Redémarrer Nginx
5 sudo systemctl restart nginx
6
7 # Recharger la configuration sans downtime
8 sudo systemctl reload nginx
9
10 # Consulter les logs d'erreur (très utile pour le débogage)
11 sudo tail -n 100 /var/log/nginx/error.log
12
13 # Consulter les logs d'accès
14 sudo tail -f /var/log/nginx/access.log
```

10.2 Vérification des ports et processus

Listing 30 – Commandes de diagnostic

```
1 # Vérifier si un port est en écoute
2 ss -tuln | grep ':8080' || echo "no process on 8080"
3
4 # Voir tous les ports en écoute
5 ss -tuln
6
7 # Vérifier le port RTMP
8 sudo ss -tuln | grep 1935
```

10.3 Gestion de Keepalived

Listing 31 – Commandes Keepalived

```
1 # Redémarrer Keepalived
2 sudo systemctl restart keepalived
3
4 # Voir le statut
5 sudo systemctl status keepalived
6
7 # Consulter les logs récents
8 sudo journalctl -u keepalived -n 50 --no-pager
9
10 # Suivre les logs en temps réel
11 sudo journalctl -u keepalived -f
12
13 # Vérifier l'IP flottante
14 ip a | grep 192.168.1.100
```

10.4 Diagnostic réseau

Listing 32 – Commandes réseau

```
1 # Voir les interfaces réseau
2 ip a
3
4 # Tester la connectivité
5 ping -c 4 <IP_destination>
6
7 # Tester une connexion TCP
8 nc -zv <IP> <port>
9
10 # Scanner les ports (depuis Kali)
11 nmap -p 80,1935,8080,8081 <IP_Ubuntu>
```

11 Conclusion

11.1 Objectifs atteints

Ce travail pratique a permis de mettre en œuvre avec succès une infrastructure réseau complète et fonctionnelle comprenant :

- Un serveur web Nginx opérationnel
- Un reverse proxy correctement configuré
- Un load balancer répartissant efficacement la charge sur deux backends
- Un serveur de streaming RTMP capable de diffuser du contenu vidéo en direct
- Un système de haute disponibilité avec basculement automatique

11.2 Compétences acquises

Durant ce TP, les compétences suivantes ont été développées :

- Configuration avancée de Nginx (reverse proxy, load balancing, module RTMP)
- Mise en place de la haute disponibilité avec Keepalived et protocole VRRP
- Diagnostic et résolution de problèmes réseau et système
- Gestion des différents modes réseau virtualisés (NAT, Bridge, Host-Only)
- Analyse de logs système pour le débogage
- Configuration de streaming vidéo avec OBS Studio
- Tests et validation d'infrastructure

11.3 Points clés à retenir

1. **Toujours vérifier les logs** : La commande `sudo tail -n 100 /var/log/nginx/error.log` a été essentielle pour résoudre les problèmes
2. **Valider les ports** : Utiliser `ss -tuln` pour s'assurer que les services écoutent sur les bons ports
3. **Tester la configuration** : Toujours exécuter `nginx -t` avant de redémarrer Nginx
4. **Adapter le réseau** : Choisir le bon mode réseau (Bridge, NAT, Host-Only) selon les besoins du protocole
5. **Documentation** : Capturer chaque étape facilite le débogage et la rédaction du rapport
6. **Haute disponibilité** : Keepalived + VRRP offrent une solution simple et efficace pour la redondance

11.4 Perspectives

Cette infrastructure constitue une base solide pour :

- Déployer des applications web en production
- Héberger des plateformes de streaming vidéo
- Créer des environnements hautement disponibles
- Servir de laboratoire pour des tests de sécurité et de performance

Les concepts appris sont directement applicables dans des contextes professionnels réels, notamment dans le domaine du DevOps, de l'administration système et de l'ingénierie réseau.

Références

- Documentation officielle Nginx : <https://nginx.org/en/docs/>
- Module RTMP Nginx : <https://github.com/arut/nginx-rtmp-module>
- Documentation Keepalived : <https://keepalived.readthedocs.io/>
- RFC 5798 - VRRP : <https://datatracker.ietf.org/doc/html/rfc5798>
- OBS Studio Documentation : <https://obsproject.com/wiki/>

A Annexe A : Fichiers de configuration complets

A.1 Configuration Nginx complète

Listing 33 – /etc/nginx/nginx.conf

```
1 user www-data;
2 worker_processes auto;
3 pid /run/nginx.pid;
4
5 events {
6     worker_connections 768;
7 }
8
9 http {
10     sendfile on;
11     tcp_nopush on;
12     types_hash_max_size 2048;
13     include /etc/nginx/mime.types;
14     default_type application/octet-stream;
15
16     # Load Balancer configuration
17     upstream my_app {
18         server 127.0.0.1:8080;
19         server 127.0.0.1:8081;
20     }
21
22     server {
23         listen 80;
24         server_name _;
25
26         location / {
27             proxy_pass http://my_app;
28             proxy_set_header Host $host;
29             proxy_set_header X-Real-IP $remote_addr;
30             proxy_set_header X-Forwarded-For
31                 $proxy_add_x_forwarded_for;
32         }
33
34         access_log /var/log/nginx/access.log;
35         error_log /var/log/nginx/error.log;
36     }
37
38     rtmp {
39         server {
40             listen 1935;
41             chunk_size 4096;
42
43             application live {
44                 live on;
```

```
45         record off;  
46     }  
47 }  
48 }
```

A.2 Configuration Keepalived MASTER

Listing 34 – /etc/keepalived/keepalived.conf (Ubuntu)

```
1 vrrp_instance VI_1 {  
2     state MASTER  
3     interface enp0s3  
4     virtual_router_id 51  
5     priority 100  
6     advert_int 1  
7     authentication {  
8         auth_type PASS  
9         auth_pass 1234  
10    }  
11    virtual_ipaddress {  
12        192.168.1.100  
13    }  
14 }
```

A.3 Configuration Keepalived BACKUP

Listing 35 – /etc/keepalived/keepalived.conf (Kali)

```
1 vrrp_instance VI_1 {  
2     state BACKUP  
3     interface eth0  
4     virtual_router_id 51  
5     priority 90  
6     advert_int 1  
7     authentication {  
8         auth_type PASS  
9         auth_pass 1234  
10    }  
11    virtual_ipaddress {  
12        192.168.1.100  
13    }  
14 }
```

B Annexe B : Scripts utiles

B.1 Script de vérification de santé

Listing 36 – check_health.sh

```

1  #!/bin/bash
2
3  echo "=== Checking Nginx ==="
4  systemctl status nginx | grep Active
5
6  echo -e "\n=== Checking Keepalived ==="
7  systemctl status keepalived | grep Active
8
9  echo -e "\n=== Checking Virtual IP ==="
10 ip a | grep 192.168.1.100
11
12 echo -e "\n=== Checking Ports ==="
13 ss -tuln | grep -E ':(80|1935|8080|8081)'
14
15 echo -e "\n=== Backend Health ==="
16 curl -s -o /dev/null -w "Backend 8080: %{http_code}\n" http
   ://127.0.0.1:8080
17 curl -s -o /dev/null -w "Backend 8081: %{http_code}\n" http
   ://127.0.0.1:8081

```

B.2 Script de test de load balancing

Listing 37 – test_loadbalancer.sh

```

1  #!/bin/bash
2
3  echo "Testing Load Balancer distribution..."
4  echo "Sending 20 requests..."
5
6  for i in {1..20}; do
7      curl -s http://127.0.0.1 | grep -o "Backend [0-9]*" >> /tmp/
        lb_test.txt
8  done
9
10 echo -e "\nResults:"
11 echo "Backend 8080: $(grep -c '8080' /tmp/lb_test.txt) requests"
12 echo "Backend 8081: $(grep -c '8081' /tmp/lb_test.txt) requests"
13
14 rm /tmp/lb_test.txt

```

B.3 Script de monitoring RTMP

Listing 38 – monitor_rtmp.sh

```

1  #!/bin/bash
2
3  echo "=== RTMP Server Status ==="
4  ss -tuln | grep 1935
5

```

```

6 echo -e "\n=== Active RTMP Connections ==="
7 ss -tn | grep 1935
8
9 echo -e "\n=== Nginx RTMP Statistics ==="
10 curl -s http://localhost/stat 2>/dev/null || echo "Stats not
    available"

```

C Annexe C : Guide de dépannage

C.1 Tableau de diagnostic rapide

Symptôme	Cause probable	Solution
502 Bad Gateway	Backend non démarré	Lancer le backend : <code>python3 -m http.server 8080</code>
Connection refused	Port fermé ou firewall	Vérifier : <code>ss -tuln</code> et <code>ufw status</code>
Interface not found	Nom interface incorrect	Vérifier avec <code>ip a</code>
Double MASTER	Réseau non L2	Utiliser mode Host-Only
RTMP ne connecte pas	Mode NAT actif	Passer en mode Bridge
VIP ne bascule pas	<code>auth_pass</code> différent	Synchroniser les configs
Nginx ne démarre pas	Erreur de syntaxe	Exécuter <code>nginx -t</code>
Logs vides	Permissions incorrectes	Vérifier <code>/var/log/nginx</code>

TABLE 1 – Guide de dépannage rapide

C.2 Commandes de diagnostic par problème

C.2.1 Problème : Nginx ne répond pas

```

1 # Vérifier le statut
2 sudo systemctl status nginx
3
4 # Vérifier les ports
5 sudo ss -tuln | grep :80
6
7 # Tester la configuration
8 sudo nginx -t
9
10 # Voir les erreurs récentes
11 sudo tail -n 50 /var/log/nginx/error.log

```

C.2.2 Problème : Backend inaccessible

```

1 # Vérifier si le backend écoute
2 ss -tuln | grep ':8080'
3
4 # Tester la connexion locale

```

```
5 curl -v http://127.0.0.1:8080
6
7 # Vérifier les processus Python
8 ps aux | grep python
```

C.2.3 Problème : Keepalived ne fonctionne pas

```
1 # Statut du service
2 sudo systemctl status keepalived
3
4 # Logs détaillés
5 sudo journalctl -u keepalived -n 100 --no-pager
6
7 # Vérifier la VIP
8 ip a | grep 192.168.1.100
9
10 # Tester la connectivité VRRP
11 sudo tcpdump -i enp0s3 vrrp
```

C.2.4 Problème : RTMP ne streame pas

```
1 # Vérifier le port RTMP
2 sudo ss -tuln | grep 1935
3
4 # Voir les connexions actives
5 sudo ss -tn | grep 1935
6
7 # Logs Nginx
8 sudo tail -f /var/log/nginx/error.log
9
10 # Tester avec ffmpeg
11 ffmpeg -re -i test.mp4 -c copy -f flv rtmp://IP/live/test
```

D Annexe D : Améliorations futures

D.1 Sécurité

- **SSL/TLS** : Implémenter HTTPS avec Let's Encrypt

```
1 sudo apt install certbot python3-certbot-nginx
2 sudo certbot --nginx -d example.com
```

- **Firewall** : Configurer UFW pour restreindre les accès

```
1 sudo ufw allow 22/tcp
2 sudo ufw allow 80/tcp
3 sudo ufw allow 443/tcp
4 sudo ufw allow 1935/tcp
5 sudo ufw enable
```

- **Fail2ban** : Protection contre les attaques brute-force

```
1 sudo apt install fail2ban
2 sudo systemctl enable fail2ban
```

D.2 Performance

- **Cache Nginx** : Activer la mise en cache

```
1 proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=my_cache
   :10m;
2 proxy_cache my_cache;
```

- **Compression** : Activer gzip

```
1 gzip on;
2 gzip_types text/plain text/css application/json;
```

- **Load Balancing avancé** : Utiliser différents algorithmes

```
1 upstream my_app {
2     least_conn; # ou ip_hash
3     server 127.0.0.1:8080;
4     server 127.0.0.1:8081;
5 }
```

D.3 Monitoring

- **Prometheus + Grafana** : Surveillance en temps réel
- **Netdata** : Monitoring système léger
- **ELK Stack** : Analyse centralisée des logs
- **Alertes email** : Notifications automatiques

D.4 Scalabilité

- Ajouter plus de serveurs BACKUP
- Utiliser Docker pour la containerisation
- Déployer avec Kubernetes pour l'orchestration
- Implémenter un DNS load balancer
- Migrer vers le cloud (AWS, Azure, GCP)